

Arayüz Tasarımı: Durum Makinesi ile Denetim Modelleme

ÖFY

September 2026

Özet

Bir arayüz düğmesi karar veren değil, olay üreten bir bileşendir; tıklandığında yalnızca bir bildirim çıkarır ve o bildirimin sonucunu bilmez. Bir olayın karşılık bulup bulmayacağını belirleyen şey düğmenin kendisi değil, olay geldiğinde sistemin içinde bulunduğu durumdur. Olayların yutulduğu haller rastgele bir liste oluşturmaz; her biri belirli bir duruma bağlıdır ve duruma bağlanmadan yazıldıklarında hem gereksiz tekrarlar hem de fark edilmeyen boşluklar barındırırlar. Bir olay yalnızca varlığıyla değil; zamanı, sırası, sıklığı, komşu olaylara uzaklığı, süresi, geldiği andaki bağlam ve başka olaylarla çakışması üzerinden de tanımlanır; bu boyutların yalnızca üçü saklanır, geri kalanı kayıttan hesaplanır. Birden fazla düğme birbirine doğrudan bağlanmaz; ortak bir duruma bağlanır ve aralarındaki karşılıklı dışlama ilişkisi ayrıca yazılmadan bu ortaklıktan türer. Durum ile olayın kesiştiği matris, ikili bayraklarla tutulan modellerin gizlediği iki şeyi açığa çıkarır: hangi kombinasyonların hiç oluşamayacağı ve hangi hallerin hiç düşünülmediği. Çift gönderim gibi klasik hatalar ek bir önlem gerektirmez; matrisin boş bırakılmış hücresi bu işlevi zaten görür. Olay akışına zaman ekleninde müdahale etmek ise mantıksal değil fiziksel bir sorunun çözümüdür ve yalnızca donanım kaynaklı tekrarlarda gerekir.

Giriş

Kullanıcı arayüzlerinde en sık karşılaşılan hata sınıfı, bir denetimin görünen hâli ile sistemin gerçek hâlinin ayrışmasıdır. Kaydet düğmesi tıklanabilir görünürken kayıt işlemi hâlihazırda sürmektedir; yükleme göstergesi dönmeye devam ederken istek çoktan tamamlanmıştır; iptal düğmesi iptal edilecek hiçbir iş kalmadığı hâlde etkin durmaktadır. Bu hataların ortak kaynağı tek tek düğmelerin yanlış programlanması değil, arayüzün hâlinin birbirinden bağımsız ikili değişkenlerle temsil edilmesidir.

Durum (*state*), sistemin herhangi bir anda içinde bulunduğu ayrık hâldir. **Olay** (*event*), sistemin dışından gelen ve bir hâl değişikliğini tetikleyebilecek bildirimdir. **Geçiş** (*transition*), belirli bir durumdayken belirli bir olay geldiğinde hangi duruma gidileceğini belirleyen kuraldır. Bu üçlünün oluşturduğu yapıya **durum makinesi** (*state machine*) adı verilir. Sonlu sayıda durum içeren hâline sonlu durum makinesi (*finite state machine*) denir ve biçimsel tanımı ardışık devrelerin sentezi üzerine yapılan çalışmalarda kurulmuştur (Mealy, 1955; Moore, 1956).

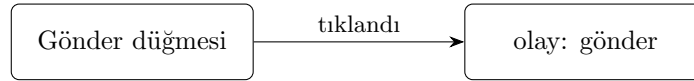
İkili bayraklarla çalışan yaygın yaklaşımın sorunu kombinatoryaldır. Beş adet doğru-yanlış değişkeni otuz iki kombinasyon üretir; bunların çoğu anlamsızdır, ancak model bu anlamsız kombinasyonların oluşmasını engelleyecek hiçbir mekanizma içermez. Anlamsız kombinasyon, ortaya çıktığında hata olarak görünür. Durum listesi ise aynı bilgiyi beş ayrık hâlde temsil eder ve geri kalan yirmi yedi kombinasyonu temsil edilemez kılar. Bu ilke yazılım tasarımında “geçersiz hâlleri temsil edilemez yapmak” biçiminde formüle edilmiştir (Minsky, 2011).

Bu not, tek bir düğmeden başlayarak iki düğmeli bir arayüzün durum modelini kurar. Kapsamı istemci tarafındaki tek bir ekranın hâl yönetimidir; dağıtık sistemlerdeki olay tabanlı mimari, kalıcı olay günlüğü ve mesaj kuyrukları kapsam dışıdır. Bu ikisi arasındaki kavramsal akrabalık ve ayrım notun sonunda ayrıca ele alınır.

Olay Üreten Bileşen Olarak Düğme

Düğmenin tek işlevi

Bir düğme tıklandığında yalnızca bir olay üretir. Bu ifadenin içerdiği kısıtlamalar, içerdiği olumlamadan daha önemlidir. Düğme isteği kendisi göndermez, kendi etkin olup olmadığına kendisi karar vermez, başka düğmelere haber vermez ve işlemin sonucunu öğrenmez.

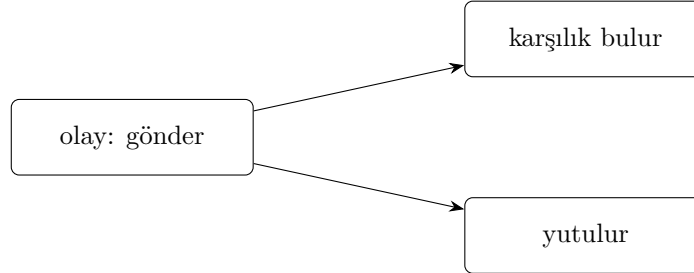


Düğmenin bir *giriş* olduğu, karar verici olmadığı buradan çıkar. Aynı tıklama, sistem bir durumdayken bir sonuç doğurur, başka bir durumdayken hiçbir sonuç doğurmaz; fark düğmede değil, düğmenin dışındadır.

Düğmenin görünümü de aynı ilkenin tersinden işleyen hâlidir. Düğme kendi etkinliğini seçmez; sistemin durumuna bakarak etkin ya da pasif görünür. Bu iki yön arasındaki ayrımın korunması kritiktir: düğme yukarı doğru olay iletir, aşağı doğru durumdan görünüm alır. Düğmenin hem olay iletip hem kendi görünümünü kendi başına değiştirdiği tasarımlarda, gerçek durum ile ekrandaki görüntü ilk beklenmedik olayda ayrışır.

Olayın iki olası sonucu

Üretilen olayın iki akıbeti vardır. Ya bir geçişle karşılaşır ve sistemin durumu değişir, ya da karşılık bulamaz ve yutulur.



Hangi dala gidileceğini belirleyen bilgi olayın kendisinde taşınmaz. Olay her seferinde aynıdır; değişen, olayın karşılaştığı bağlamdır.

Yutulma halleri

Gönder olayının yutulduğu haller dört başlıkta toplanır: işlem hâlihazırda sürmektedir, form doğrulamadan geçmemiştir, bulunulan durumda böyle bir geçiş tanımlı değildir, gönderilecek yeni bir değişiklik yoktur.

Bu dördü kullanıcı açısından ayırt edilemez; hepsinde tıklama hiçbir şey yapmamış görünür. Sistem açısından ise aynı sınıfta değildirler. İlk ikisi geçicidir:

koşul değıştiğinde olay karşılık bulmaya başlar. Üçüncüsü yapısaldır: o durumda böyle bir geçiş hiç tanımlanmamıştır ve ne kadar beklenirse beklensin tanımlanmayacaktır. Dördüncüsü ise geçişin var olduğu, koşulun da sağlandığı, ancak yapılacak işin bulunmadığı hâldir.

İptal olayı için aynı çerçeve kurulduğunda simetrik bir liste çıkar: iptal edilecek iş yoktur, iş tamamlanmıştır ve geç kalınmıştır, bulunulan durumda geçiş tanımlı değildir, iptal işlemi hâlihazırda sürmektedir. İki liste arasındaki yapısal fark şudur: gönder olayının yutulma gerekçeleri ağırlıklı olarak “henüz olmadı” biçimindeyken, iptal olayının kiler “artık geç” biçimindedir.

Bu listelerin serbest hâlde tutulması iki sorun üretir. Birincisi tekrar: “geçiş tanımlı değil” maddesi ayrı bir gerekçe değil, geçiş tablosundaki bir hücrenin boş bırakılmış olmasının başka bir ifadesidir. İkincisi eksik: iptal listesindeki “zaten iptal ediliyor” maddesi, durum listesinde bir **iptal ediliyor** hâli bulunmasını gerektirir. Böyle bir hâl tanımlanmamışsa madde ya uydurmadır ya da eksik bir durumu işaret eder. Serbest liste bu tutarsızlığı gizler; duruma bağlanmış tablo açığa çıkarır.

Olayın Ölçülebilir Boyutları

Sekiz eksen

Bir olay yalnızca gerçekleşip gerçekleşmediğiyle tanımlanmaz. Üzerinde işlem yapılabilecek sekiz boyut taşır.

Varlık, olayın oluşup oluşmadığıdır. İkili değer alır ve en temel eksenidir.

Zaman, olayın hangi anda gerçekleştiğidir. Tek bir sayıyla ifade edilir ve olayın mutlak konumudur.

Sıra, olayın hangi olaydan sonra geldiğidir. Zamanın ölçüsüz karşılığıdır; iki olay arasında kaç milisaniye geçtiği bilinmese dahi hangisinin önce geldiği bilinebilir.

Sıklık, birim zamanda kaç olay gerçekleştiğidir. Konumun türevine karşılık gelir.

Aralık, ardışık iki olay arasındaki mesafedir. Sıklığın tekil hâlidir.

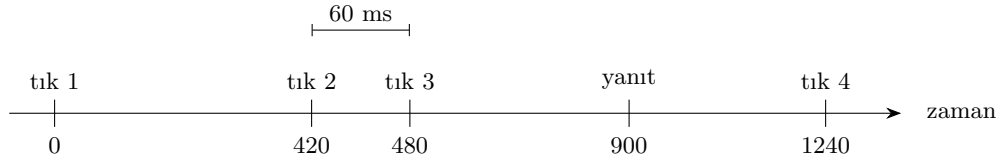
Süre, olayın kendisinin uzunluğudur. Tıklama anlık olduğu için süresi sıfırdır; basılı tutma ise başlangıcı ve bitişi olan bir olaydır.

Bağlam, olay geldiğinde sistemin hangi durumda bulunduğuudur. Aynı olay farklı durumlarda farklı anlam taşır.

Eşzamanlılık, olayla aynı anda başka bir olayın bulunup bulunmadığıdır. Çakışma eksenini olarak da adlandırılabilir.

Eksenlerin somut karşılığı

Aşağıdaki tıklama dizisi bu eksenlerin nasıl okunduğunu gösterir. Zamanlar milisaniye cinsindendir ve dördüncü satırdaki yanıt olayı kullanıcıdan değil sistemden gelmiştir.



İkinci ve üçüncü tıklama arasındaki altmış milisaniyelik aralık, bağımsız bir insan kararının oluşabileceğinden kısadır. Bu nedenle üçüncü tıklama yeni bir istek değil, aynı isteğin tekrarı olarak yorumlanabilir. Dördüncü tıklama ise yanıt olayından sonra gelmiştir; öncekilerle aynı düğmeye basılmış olmasına rağmen taşıdığı anlam farklıdır. Üçüncü tıklama sabırsızlık göstergesiye, dördüncü tıklama bilinçli bir yeni istektir. Bu ayrımı üreten şey tıklamanın kendisi değil, komşu olaylara göre konumudur.

Veri modeli

Sekiz eksenin tamamı ayrı ayrı saklanmaz. Kayıt yalnızca dört alan içerir; geri kalan boyutlar bu kayıttan hesaplanır.

Kimlik	Kaynak	Zaman	Durum
1	gönder	0	boş
2	gönder	420	gönderiliyor
3	gönder	480	gönderiliyor
4	iptal	900	gönderiliyor
5	gönder	1240	hata

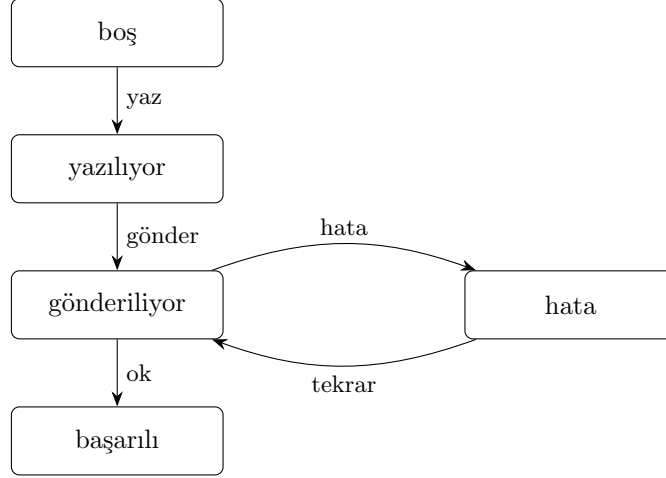
Türetilen boyutlar şunlardır: sıra satır numarasından, aralık bir satırın zamanı ile bir öncekinin zamanı arasındaki farktan, sıklık belirli bir zaman penceresine düşen satır sayısından, varlık satırın bulunup bulunmamasından, eşzamanlılık ise zaman aralıkları örtüşen satırlardan elde edilir. Süre için ek bir alan gerekir ve yalnızca basılı tutma gibi uzunluğu olan olaylarda anlamlıdır; bu durumda başlangıç ve bitiş ayrı yazılır.

Sonradan hesaplanamayan tek boyut bağlamdır. Olayın geldiği andaki durum o anda kaydedilmezse geriye dönük olarak bulunamaz, çünkü durum değişmeye devam eder ve geçmiş hâli kayıt dışında hiçbir yerde tutulmaz. Bir tıklamanın neden sonuçsuz kaldığı sorusunun cevabı yalnızca bu alandan okunur. Bu nedenle olay kaydına her zaman o anki durum yazılır.

Durum Makinesi

Durumlar ve geçişler

Form gönderme akışı beş ayrık durum içerir: **boş**, **yazılıyor**, **gönderiliyor**, **başarılı**, **hata**. Geçişler şu biçimde okunur.



Bu modelde bir düğmenin etkin olup olmadığı ayrı bir değişkende tutulmaz, durumun bir fonksiyonu olarak hesaplanır. Sistem **gönderiliyor** durumundaysa gönder düğmesi pasiftir; bunun için ek bir bayrak gerekmez ve dolayısıyla o bayrağın güncellenmeyi unutulması ihtimali de ortadan kalkar.

Geçişin üzerindeki bilgiler

Geçiş yalnızca bir ok değildir. Üzerinde dört bilgi taşır: hangi durumdan çıkıldığı, hangi olayla tetiklendiği, hangi koşul sağlanırsa geçerli olduğu ve geçiş sırasında hangi işin yapıldığı.

Koşul (*guard*), aynı olayın farklı yerlere götürülmesini sağlar. Gönder olayı, form doğrulamadan geçtiyse **gönderiliyor** durumuna, geçmediyse **hata** durumuna yönlendirilir. Olay aynıdır, koşul farklıdır, varış noktası farklıdır.

Zaman geçişe iki biçimde girer. Birincisi olay olarak: belirli bir sürenin dolması da bir olaydır ve geçişi tetikleyebilir. **gönderiliyor** durumundan otuz saniye sonra **hata** durumuna geçilmesi bu türdendir; tetikleyen kullanıcı değil saattir. İkincisi süre olarak: bazı durumlar anlık değil aralıklıdır ve içinde geçirilen zaman kendi başına bilgi taşır.

Öncelik, aynı anda birden fazla olayın gelmesi hâlinde hangisinin kazanacağını belirler. Sunucu yanıtı ile kullanıcının iptal talebi çakıştığında iptalin kazanması gerekir; aksi hâlde kullanıcının iptal ettiği işlem başarılı olarak görünür. İç içe durumlarda da öncelik geçerlidir: dıştaki durumda tanımlı bir geçiş, içteki durumda tanımlı olanı ezer.

Eylem, geiş sırasında yrtlen iřtir. İsteęin gnderilmesi, gnlęe yazılması, form alanlarının temizlenmesi bu trdeendir. Eylem durumun parası deęildir; geişin zerine asılmıř bir yan etkidir.

Bu bilgilerin geiş tablosuna yazılmaması, kod iine daęılmaları anlamına gelir. Daęıldıklarında “řu anda hangi geişler mmkn” sorusunun cevabı tek bir yerden okunamaz hle gelir.

İki Düğmenin İlişkisi

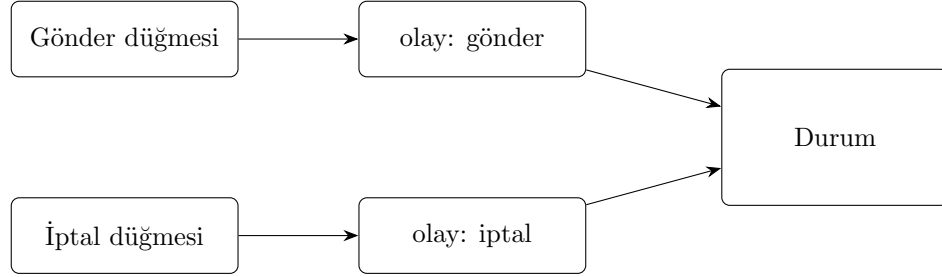
Doğrudan bağlamanın sorunu

Gönder ve iptal düğmeleri arasında sezgisel olarak bir ilişki kurulmak istenir: biri etkinken diğeri pasif olmalıdır. Bu ilişkinin düğmeden düğmeye doğrudan bağlantı olarak kurulması yaygın ancak hatalı bir çözümdür.

Hatanın iki nedeni vardır. Birincisi ölçeklenme sorunudur: N düğme arasında doğrudan ilişki kurulduğunda tanımlanması ve bakımı gereken bağlantı sayısı N^2 ile büyür. İkincisi tutarlılık sorunudur: her düğmenin etkinliği ayrı bir yerde tutulduğunda, bir durum değişikliğinde bunların hepsinin güncellenmesi gerekir. Biri unutulduğunda ekranda birbirleriyle çelişen iki düğme görünür ve bu hata ancak belirli bir olay sırasında ortaya çıktığı için kolay fark edilmez.

Ortak durum üzerinden dolaylı ilişki

Doğru kurgu, iki düğmeyi birbirine değil ortak bir duruma bağlamaktır.



Bu kurguda düğmeler birbirinden habersizdir. Her biri kendi olayını üretir, her olay aynı karar merkezine ulaşır ve karar tek yerden verilir. Düğmeler arasındaki ilişki tanımlanmaz; ortak duruma bağlı olmalarının bir sonucu olarak doğar.

Durum ve olay matrisi

İlişkinin somut hâli, satırları durumlar ve sütunları olaylar olan bir matristir. Her hücre, o durumda o olay geldiğinde hangi duruma gidileceğini bildirir. Boş hücre, geçişin tanımlı olmadığını ve olayın yutulacağını gösterir.

Durum	gönder	iptal
boş	—	—
yazılıyor	gönderiliyor	—
gönderiliyor	—	boş
başarılı	—	—
hata	gönderiliyor	—

Boş hücrelerin gerekçeleri, önceki bölümde serbest liste hâlinde verilen yutulma hallerinin duruma dağıtılmış karşılığıdır. **boş** durumunda gönder olayı “gönderilecek değişiklik yok” gerekçesiyle, **yazılıyor** durumunda “form geçersiz” gerekçesiyle, **gönderiliyor** durumunda “işlem hâlihazırda sürüyor” gerekçesiyle yutulur. İptal olayı ise **boş** ve **yazılıyor** durumlarında “iptal edilecek iş yok”, **başarılı** durumunda “iş tamamlandı, geç kaldı” gerekçesiyle yutulur.

Matrisin okuttuğu üç şey

Matris, serbest listenin göstermediği bilgileri görünür kılar.

Hiçbir satırda iki hücre birden dolu değildir. Bu, gönder ve iptal düğmelerinin hiçbir zaman aynı anda etkin olmayacağı anlamına gelir. Söz konusu kural hiçbir yerde ayrıca yazılmamıştır; iki düğmenin aynı duruma bağlanmasından türemiştir. Doğrudan bağlama yaklaşımında aynı garanti elde edilemez, yalnızca elde edildiği umulur.

başarılı satırının tamamen boş olması, bu duruma girildikten sonra mevcut iki düğmeyle hiçbir yere gidilemeyeceğini gösterir. Sistem oraya girip kalır. Bu, üçüncü bir çıkışın tanımlanması gerektiğini işaret eder; örneğin yeni bir form başlatan bir olay.

hata satırında iptal hücresinin boş olması, hata durumunda iptalin ne yapacağını hiç düşünülmediğini gösterir. Bu boşluk bilinçli bir karar da olabilir, unutulmuş bir hâl de. Ayrım önemlidir ve yalnızca matris kurulduğunda sorulabilir hâle gelir. Serbest liste hâlinde bu soru hiç oluşmaz.

Hızlı tıklamanın matristeki karşılığı

Bir düğmeye arka arkaya hızlı basılması ayrı bir problem sınıfı oluşturmaz. Birinci tıklama **yazılıyor** satırından geçer ve durumu **gönderiliyor** yapar. İkinci ve üçüncü tıklamalar artık **gönderiliyor** satırına düşer; o hücre boş olduğu için yutulurlar. Çift gönderim hatası için ek bir önlem gerekmez, matris bu işlevi zaten görür.

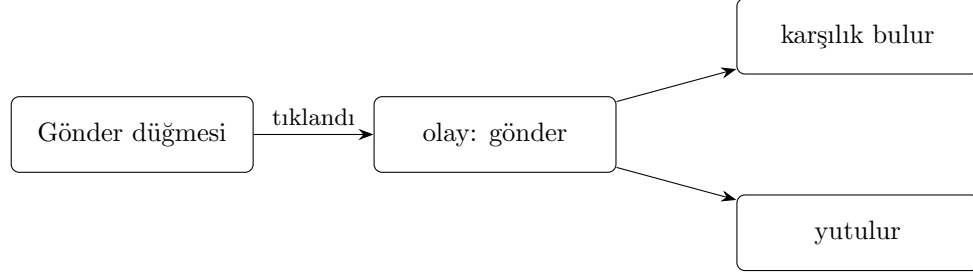
Matrisin çözemediği tek hâl, ilk tıklamanın durumu değiştirmesine fırsat kalmadan ikincisinin gelmesidir. Bu genellikle fiziksel tuş titremesi gibi donanım kaynaklı tekrarlardan doğar ve mantıksal bir sorun değildir. Çözümü de mantık katmanında değil, olay akışında yer alır: belirli bir eşiğin altındaki aralıklarla gelen ikinci olay sisteme ulaşmadan filtrelendir. Zaman ekseninde yapılan bu müdahale, olayların geciktirilmesi, seyreltilmesi, birleştirilmesi ya da sıraya alınması biçimlerini alabilir.

İki koruma birbirinin yerine geçmez. Matris mantıksal koruma sağlar ve bu durumda bu olayın anlamsız olduğunu söyler. Akış filtresi fiziksel koruma sağlar ve bu olayın gerçek bir kullanıcı kararı bile olmadığını söyler. İkincisine ihtiyaç seyrek; matrisin doğru kurulması çoğu durumda yeterlidir.

Modelin Adım Adım Kurulması

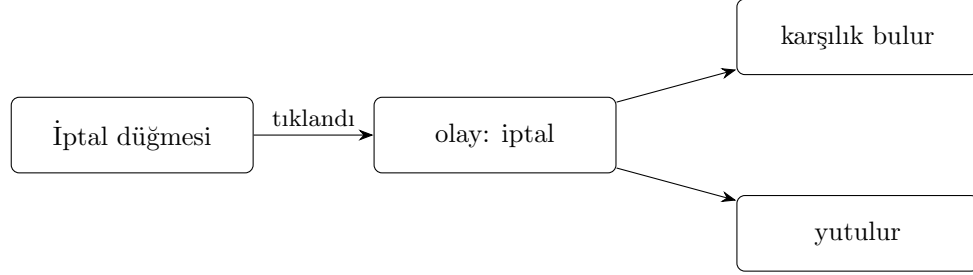
Birinci adım: gönder düğmesi

Gönder düğmesi tıklandığında bir olay üretir. Üretilen olay ya bir geçişle karşılaşır ya da yutulur.



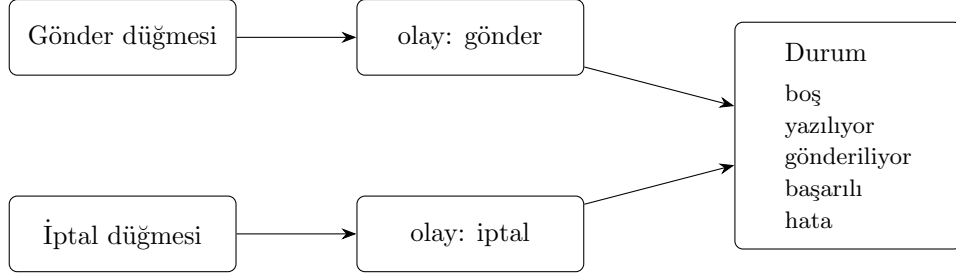
İkinci adım: iptal düğmesi

İptal düğmesi aynı yapıyı taşır. Tek fark ürettiği olayın adıdır; dallanma biçimi ve dalların anlamı değişmez.



Üçüncü adım: ortak duruma bağlanma

Her iki hattın dallanma kararını veren merci aynıdır. Düğmeler birbirine değil, tek bir duruma bağlanır ve o durumun barındırdığı hâller her iki olay için ortak referans oluşturur.



Dördüncü adım: matrise dökme

Satırlar durumun hâllerini, sütunlar olayları gösterir. Her hücre, o durumda o olay geldiğinde hangi duruma gidileceğini bildirir; dolu hücre geçişin tanımlı olduğu, tire ise olayın yutulduğu anlamına gelir.

Durum	gönder	iptal
boş	—	—
yazılıyor	gönderiliyor	—
gönderiliyor	—	boş
başarılı	—	—
hata	gönderiliyor	—

Matrisin okuttuğu üç şey vardır.

Hiçbir satırda iki hücre birden dolu değildir. Gönder etkinken iptalin pasif olması gerektiği yolundaki kural hiçbir yerde yazılmamıştır; iki düğmenin aynı duruma bağlanmasının sonucu olarak ortaya çıkmıştır.

başarılı satırı tamamen boştur. O duruma girildikten sonra mevcut iki düğmeyle hiçbir yere gidilemez. Bu, üçüncü bir çıkışın gerektiğini gösterir; yeni bir form başlatan bir olay bu işlevi görebilir.

hata satırında iptal hücresi boştur. Hata durumunda iptalin ne yapacağı hiç kararlaştırılmamıştır. Bu hücre bilinçli olarak da boş bırakılmış olabilir, gözden kaçmış da olabilir; ayrımın sorulabilmesi matrisin kurulmasına bağlıdır. Gereçekler serbest liste hâlinde tutulduğunda bu boşluk görünmez.

Yakın Kavramlarla İlişki ve Sınırlar

Hiyerarşi ve paralellik

Düz bir durum makinesi, gerçek ekranların karmaşıklığı arttıkça yetersiz kalır. Birbirinden bağımsız iki boyut eklendiğinde durum sayısı toplanarak değil çarpılarak büyür. Bu soruna karşı geliştirilen biçimsel çerçeve, durum makinelerine iç içe geçme ve paralel bölge kavramlarını eklemiştir (Harel, 1987).

İç içe geçme, birbiriyle ilişkili durumların bir üst durumun altında toplanmasıdır. Oturum açık hâlinin altında gezinme, düzenleme ve kaydetme alt durumları bulunabilir; oturumu kapatan geçiş üst düzeyde bir kez tanımlanır ve bütün alt durumlar için geçerli olur.

Paralel bölge, aynı anda birbirinden bağımsız çalışan eksenlerin ayrı tutulmasıdır. Bağlantı durumunun çevrimiçi ya da çevrimdışı olması ile form durumunun düzenleniyor ya da kaydediliyor olması bağımsız eksenlerdir. Ayrı bölgeler olarak modellendiklerinde durum sayısı çarpım yerine toplam olarak büyür.

Olay tabanlı mimariden ayrımı

Bu notta kurulan model ile dağıtık sistemlerdeki olay tabanlı mimari (*event driven architecture*) arasında gerçek bir akrabalık vardır. İkisinde de olay birinci sınıf bir kavramdır ve olayı üreten taraf sonucunu bilmez. Düğme kendi sonucunu bilmez, yayın yapan servis kendi dinleyicilerini bilmez. Temel hamle aynıdır: olayı üretenden ayırmak.

Ayrım üç noktada belirginleşir. Bu notta kurulan modelde tek bir karar merkezi vardır, olay tabanlı mimaride ise olayı kaç bileşenin dinlediği bilinmez. Bu modelde işleme eşzamanlıdır ve sonuç anında belirlenir, olay tabanlı mimaride ise araya kuyruk girer ve işleme eşzamansızdır. Bu modelde olay işlendikten sonra kaybolur, olay tabanlı mimaride ise kalıcı olarak saklanabilir ve sonradan yeniden oynatılabilir.

Bu nedenle kurulan yapının adı olay tabanlı mimari değil, durum makinesidir. İstemci tarafındaki yaygın karşılığı, olayların yukarı doğru, durumun aşağı doğru aktığı tek yönlü veri akışı kalıbıdır.

Modelin maliyeti

Durum makinesi kurmanın bir bedeli vardır ve her arayüz bu bedeli hak etmez. Üç denetimden oluşan, eşzamansız işlem içermeyen bir ekranda geçiş tablosu yazmak gereksiz yüküdür.

Modelin değeri şu koşullarda ortaya çıkar: eşzamansız işlem bulunması, iptal edilebilir bir işin olması, çok adımlı bir akışın izlenmesi, ya da “şu anda ne oluyor” sorusunun cevabının birden fazla değişkene birden bakmayı gerektirmesi. Bu koşullardan hiçbiri yoksa model, çözdüğü sorundan daha fazla karmaşıklık ekler.

Sonuç

Arayüz hatalarının önemli bir kısmı, temsil edilmemesi gereken hâllerin temsil edilebilir bırakılmasından doğar. Bağımsız ikili değişkenler bu hâlleri üretirken, ayrık durum listesi onları baştan imkânsız kılar. Aradaki fark hata yakalama kabiliyeti değil, hatanın oluşabileceği uzayın büyüklüğüdür.

Birden fazla denetim arasındaki ilişkinin ortak duruma bağlanma yoluyla dolaylı kurulması, karşılıklı dışlama gibi garantilerin ayrıca yazılmadan elde edilmesini sağlar. Yazılmayan kural ihlal edilemez; buna karşılık her denetim için ayrı tutulan bayraklar, güncellenmeyi unutuldukları ölçüde ihlal edilir.

Durum ile olayın kesiştiği matrisin asıl değeri, doldurulan hücrelerde değil boş bırakılanlardadır. Boş hücre iki farklı şeyi işaret edebilir: o geçişin bilinçli olarak tanımlanmadığını ya da hiç düşünülmendiğini. Matris bu ikisini birbirinden ayırmaz, ancak ayırmanın sorulabilmesini sağlar. Serbest liste hâlinde tutulan gerekçelerde bu soru hiç oluşmaz; eksiklik ancak çalışma zamanında bir hata olarak görünür.

Çerçevenin sınırları üç noktada belirgindir. Birincisi, durum sayısı arttıkça düz model yetersiz kalır ve hiyerarşi ile paralel bölgelere ihtiyaç doğar. İkincisi, donanım kaynaklı tekrarlar mantık katmanında çözülemez ve olay akışında ayrı bir müdahale gerektirir. Üçüncüsü, basit arayüzlerde modelin kendisi çözdüğü sorundan daha maliyetlidir.

Açık kalan sorulardan biri, bağlam alanının kaydedilmesinin getirdiği maliyetlerdir. Her olayla birlikte o anki durumun yazılması, yüksek hacimli sistemlerde kayıt boyutunu artırır; hangi durumlarda bu maliyetin gerekçelendirilebileceği ayrıca incelenmeyi gerektirir. İkinci bir açık soru, kullanıcıya verilen geri bildirimin modelde nerede duracağıdır: bir olayın yutulduğu bilgisi, yutulma gerekçesiyle birlikte kullanıcıya ne ölçüde gösterilmelidir. Bu notta kurulan çerçeve sorunun nerede sorulacağını belirler ancak cevabını içermez.

Kaynaklar

- Harel, D. (1987). Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3).
- Mealy, G. H. (1955). A Method for Synthesizing Sequential Circuits. *Bell System Technical Journal*, 34(5).
- Minsky, Y. (2011). Effective ML. Jane Street Tech Talks. “Make illegal states unrepresentable” ilkesinin bu konuşmadaki formülasyonu esas alınmıştır.
- Moore, E. F. (1956). Gedanken-Experiments on Sequential Machines. *Automata Studies*, Princeton University Press.